

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



T3.2 Estructuras de datos



Resultados de aprendizaje

- a. Conocer las 5 estructuras principales de Redis (Strings, Hashes, Lists, Sets, Sorted Sets)
- b. Saber elegir la estructura adecuada según el caso de uso
- c. Comprender el patrón embeber vs referenciar y cuándo aplicar cada opción
- d. Explorar módulos avanzados como RediSearch y RedisJSON

Índice de contenidos

T3.2.1 ¿Qué estructura de datos uso?	T3.2.6 Lists — Listas enlazadas
T3.2.2 Embeber vs Referenciar datos	T3.2.7 Sets — Conjuntos únicos
T3.2.3 Modelado de feeds y timelines	T3.2.8 Sorted Sets — Conjuntos ordenados
T3.2.4 Strings — El tipo básico	T3.2.9 Resumen visual de estructuras de datos
T3.2.5 Hashes — Campos y valores	T3.2.10 Redis Modules — Ampliando las capacidades

T3.2.1 ¿Qué estructura de datos uso?

Antes de profundizar en cada estructura, esta tabla te ayuda a decidir cuál usar según tu necesidad:

Necesidad	Estructura	Ejemplo	Por qué
Almacenar un valor simple con TTL	String	SET cache:product:456 "data" EX 300	O(1), mínima memoria, expiración automática
Objeto con múltiples campos que se actualizan individualmente	Hash	HSET user:1000 name "Juan"	Actualización parcial sin leer todo, ~30% menos memoria que JSON
Cola FIFO (primero en entrar, primero en salir)	List	LPUSH + BRPOP	Inserción/eliminación O(1) por extremos
Pila LIFO (primero en entrar, último en salir)	List	LPUSH + LPOP	Misma estructura, diferente uso
Conjunto sin duplicados	Set	SADD likes:post:1 "user:1000"	O(1) membresía, operaciones de conjuntos
Ranking / leaderboard / timeline	Sorted Set	ZADD leaderboard 1500 "player:1"	Ordenado por score, rangos O(log N)
Contador atómico (vistas, likes)	String (numérico)	INCR views:product:456	O(1), atómico bajo concurrencia
Geolocalización	Geo	GEOADD locations 2.35 48.85 "Paris"	Búsqueda por radio, distancia
Bit manipulation (flags, features)	Bitmap	SETBIT user:1000:features 0 1	Muy eficiente en memoria para flags

Regla práctica: Si solo necesitas almacenar un valor con TTL → **String**. Si necesitas campos independientes → **Hash**. Si necesitas ordenar por puntuación → **Sorted Set**. En caso de duda, empieza por String y migra si necesitas más funcionalidad.

T3.2.2 Embeber vs Referenciar datos

En Redis, una decisión clave de modelado es **cuándo embeber datos dentro de una clave** (Hash) vs **usar claves separadas** (referencias):

Opción A: Embeber (Hash)

```
# Un Hash con todos los campos del usuario
HSET user:1000:profile
  nombre "Juan García"
  email "juan@email.com"
  ciudad "Madrid"
  telefono "+34 600 123 456"
  fecha_registro "2025-01-15"

# Acceso parcial eficiente
HGET user:1000:profile nombre # Solo "Juan García"
HGETALL user:1000:profile      # Todos los campos
```

Opción B: Referenciar (claves separadas)

```
# Claves separadas por campo
SET user:1000:nombre "Juan García"
SET user:1000:email "juan@email.com"
SET user:1000:ciudad "Madrid"
SET user:1000:telefono "+34 600 123 456"
SET user:1000:fecha_registro "2025-01-15"

# Acceso individual
GET user:1000:nombre # Solo "Juan García"
# Para obtener todo: múltiples GET (menos eficiente)
```

Criterio	Embeber (Hash)	Referenciar (claves separadas)
Memoria	~30% menos (un solo prefijo de clave)	Más (prefijo por campo)
Acceso parcial	Sí — HGET lee solo un campo	Sí — GET lee solo una clave
Acceso completo	Óptimo — HGETALL en una llamada	Costoso — múltiples GET roundtrips
TTL por campo	No — TTL a toda la clave	Sí — cada clave tiene su TTL
Actualización parcial	HSET solo cambia un campo	SET solo cambia una clave
Atomicidad	HSET/HGET no atómicos entre campos	MULTI/EXEC para atomicidad
Escalabilidad	Un nodo almacena todo el Hash	Claves pueden distribuirse en clúster

Regla práctica: Embebe datos que siempre se leen juntos (perfil de usuario). Referencia datos con TTLs diferentes o que escalan de forma independiente (sesión vs perfil vs configuración).

Regla práctica: Si un objeto tiene 5 campos y siempre se leen todos juntos → Hash. Si tiene 1 campo con TTL 30 minutos y otro con TTL 24 horas → claves separadas porque cada una necesita su propia expiración.

Concepto clave: Para modelar datos en Redis, primero pregunta: ¿Se leen siempre juntos? → Embebe (Hash). ¿Tienen TTLs diferentes? → Referencia (claves separadas). ¿Necesitas atomicidad? → MULTI/EXEC o Lua scripts.

 **Instagram (modelado real):** Instagram embebe datos de perfil en Hashes (nombre, bio, avatar) porque se leen siempre juntos. Pero usa claves separadas para contadores (followers, posts) porque tienen TTLs diferentes y escalan de forma independiente.

T3.2.3 Modelado de feeds y timelines

Uno de los casos de uso más comunes de Redis en redes sociales es modelar **feeds de noticias y timelines**. Veamos cómo Twitter e Instagram lo hacen:

Twitter: Timeline de usuario

Timeline de Twitter – Sorted Sets

```
# Cuando un usuario publica un tweet
# 1. Añadir el tweet a su timeline personal
ZADD user:1000:timeline 1726500000 "tweet:abc123"

# 2. Enviar el tweet a los timelines de sus seguidores
# (operación fan-out en escritura)
ZADD user:2000:timeline 1726500000 "tweet:abc123"
ZADD user:3000:timeline 1726500000 "tweet:abc123"
ZADD user:4000:timeline 1726500000 "tweet:abc123"

# 3. Leer la timeline (los 50 tweets más recientes)
ZREVRANGE user:1000:timeline 0 49 WITHSCORES
# → ["tweet:xyz789", "tweet:abc123", ...]

# 4. Tweets de los últimos 30 minutos
ZREVRANGEBYSCORE user:1000:timeline $(date +%s) $(($(date +%s) - 1800))
```

 **Twitter (datos reales):** Twitter almacena cada timeline como un Sorted Set con el timestamp como score. Un usuario popular puede tener millones de seguidores — el fan-out en escritura es costoso, por eso Twitter usa Redis como caché de escritura y L3 como respaldo.

Instagram: Feed de noticias

Feed de Instagram – Patrón híbrido

```
# Instagram usa un patrón híbrido:
# 1. Sorted Set para el feed (orden cronológico)
ZADD user:5000:feed 1726500000 "post:post123"

# 2. Hash para los datos del post (embebido)
HSET post:post123
  autor "ana_foto"
  likes 1247
  comentarios 89
  media_url "https://cdn.instagram.com/post123.jpg"
  fecha "2025-09-16T10:00:00Z"

# 3. Set para "quién ya vio el post" (evitar duplicados)
SADD post:post123:vistos 5000 5001 5002

# 4. Contador de Likes atómico
INCR likes:post:post123
```

 **Instagram (datos reales):** Instagram genera ~100M de likes al día. Cada like es un INCR atómico en Redis. El feed usa Sorted Sets con TTL de 24h — los posts más viejos se eliminan automáticamente de la caché.

¿Fan-out en escritura o en lectura?

Patrón	Cómo funciona	Ventajas	Inconvenientes
Escritura (push)	Al publicar, copiar el post a todos los timelines de seguidores	Lectura rápida (O(1))	Escritura costosa (N copias por N seguidores)
Lectura (pull)	Al leer, buscar los posts de quienes sigues	Escritura barata (1 solo post)	Lectura lenta (buscar N seguidores)
Híbrido	Escritura push + cache TTL. Usuarios populares usan pull	Balance entre ambos	Complejidad de implementación

Redis en feeds: Sorted Sets son la estructura natural para timelines (score = timestamp). Los contadores de likes/comments usan INCR atómico. Los Set se usan para deduplicación (quién ya vio/reaccionó). El patrón híbrido (push + cache TTL) es el más utilizado en producción.

T3.2.4 Strings — El tipo básico

Los strings son el tipo de dato más simple y versátil en Redis. Pueden contener texto, números, bytes e incluso datos serializados (JSON, XML, etc.). Internamente, Redis almacena strings de tres formas según su contenido:

Contenido avanzado / opcional: Los detalles sobre la representación interna de Redis (listpack, hash table, skip list) son interesantes para entender el rendimiento, pero no son necesarios para usar Redis correctamente. Si el ritmo de clase va rápido, puedes profundizar aquí. Si no, continúa con la siguiente sección.

- **int:** Si el string es un número entero (hasta $2^{32}-1$). Ocupa muy poca memoria.
- **embstr:** Strings pequeños (< 44 bytes). Se almacenan junto al descriptor en un solo bloque de memoria.
- **raw:** Strings grandes (> 44 bytes). Se almacenan en un bloque separado.

Operaciones con Strings

```
# === Almacenamiento básico ===
SET user:1000:name "Juan García"
GET user:1000:name           # "Juan García"

# === Strings numéricos ===
SET product:456:views 0
INCR product:456:views      # 1
INCR product:456:views      # 2
INCRBY product:456:views 10 # 12
DECR product:456:views      # 11
DECRBY product:456:views 5  # 6

# === Strings con expiración ===
SETEX session:abc123 1800 "userId:1000" # TTL 30 min
SET cache:product:456 "iPhone" EX 300    # TTL 5 min
SET lock:resource1 "owner-123" NX EX 10   # Lock con TTL 10s

# === Operaciones avanzadas ===
APPEND user:1000:name " García" # Añadir al final
STRLEN user:1000:name           # Longitud: 14
SETRANGE user:1000:name 0 "Pedro" # Sobreescribir desde posición 0
GETRANGE user:1000:name 0 4       # Substring: "Pedro"

# === Lectura/escritura múltiple (más eficiente que N gets) ===
MSET user:1000:name "Juan" user:1000:email "juan@email.com"
MGET user:1000:name user:1000:email
```

¿Cuándo usar strings vs hashes?

Si tu objeto tiene **un solo campo**, usa un string. Si tiene **múltiples campos** que se actualizan individualmente, usa un hash.

GitHub usa INCR para contadores de rate limiting por usuario (5 000 peticiones/hora autenticadas). Cada usuario tiene una clave `ratelimit:{user_id}:{timestamp_hour}` que se incrementa con cada petición y expira automáticamente tras una hora.

Resumen Strings: Tipo básico y versátil. Internamente se almacena como int, embstr (<44 bytes) o raw. Usa INCR para contadores atómicos, SET EX para TTL, y MSET/MGET para operaciones múltiples más eficientes.

T3.2.5 Hashes — Campos y valores

Los hashes almacenan pares **campo-valor** dentro de una clave. Son ideales para representar objetos con múltiples propiedades (perfiles de usuario, productos, configuraciones).

Internamente: listpack vs hash table

Redis almacena hashes de dos formas según su tamaño:

- **listpack (antes ziplist):** Cuando el hash tiene pocos campos (< 128) y cada campo es pequeño (< 64 bytes). Es una estructura compacta que ahorra memoria.
- **hash table:** Cuando el hash crece más allá de esos umbrales. Usa una hash table clásica con rehashing incremental.

¿Qué es un listpack? Es una estructura de datos compacta que almacena múltiples entradas en un solo bloque contiguo de memoria. En lugar de usar punteros por cada elemento (que gastan 8 bytes cada uno), codifica todo secuencialmente. Ventaja: mínimo overhead de memoria. Desventaja: para listas grandes, la inserción media es $O(N)$ porque hay que desplazar elementos.

Operaciones con Hashes

```
# === Crear hash con múltiples campos ===
HSET user:1000 name "Juan García"
HSET user:1000 email "juan@email.com"
HSET user:1000 age 28
HSET user:1000 city "Madrid"

# Crear hash de una vez (más eficiente)
HSET user:1001 name "Ana López" email "ana@email.com" age 25

# === Leer campos ===
HGET user:1000 name           # "Juan García"
HMGET user:1000 name email    # ["Juan García", "juan@email.com"]
HGETALL user:1000             # Todos los campos y valores
HKEYS user:1000               # Solo los nombres de campo
HVALS user:1000               # Solo los valores

# === Operaciones sobre campos ===
HEXISTS user:1000 email       # 1 (existe)
HLEN user:1000                # 4 (número de campos)
HDEL user:1000 city           # Eliminar campo

# === Incrementar campo numérico ===
HINCRBY user:1000 age 1       # 29
HINCRBYFLOAT user:1000 salary 500.50

# === Búsqueda de campos ===
HSCAN user:1000 0 MATCH na* COUNT 10 # Buscar campos que empiezan por "na"
```

¿Cuándo usar hashes vs strings serializados?

Aspecto	Hash	String serializado (JSON)
Actualización parcial	Sí (solo un campo con HSET)	No (leer → deserializar → modificar → serializar → escribir)
Memoria	Más eficiente (small hash encoding)	Mayor overhead (JSON parsing)
Lectura completa	Una operación (HGETALL)	Una operación (GET)
Lectura parcial	Óptima (HGET)	Ineficiente (leer todo y descartar)
Legibilidad	Campos individuales	JSON completo
Uso de memoria	~30% menos que JSON	Más overhead por metadatos JSON

«No pongas todo en una sola cesta.» — Proverbio. En Redis, elegir la estructura de datos adecuada es como elegir el recipiente adecuado: un Hash para datos relacionados, un Sorted Set para rankings, un Set para colecciones sin duplicados.

Instagram almacena datos de perfil de usuario en hashes (`user:123:profile` → `{name, bio, avatar_url}`), permitiendo actualizaciones parciales sin leer el documento completo. Cuando un usuario cambia solo su bio, se ejecuta `HSET user:123:profile bio "Nueva bio"` en lugar de leer todo el perfil, modificar y escribir de nuevo.

Resumen Hashes: Mapa campo-valor dentro de una clave. Usa HSET/HGET para acceso parcial. Internamente: listpack (pocos campos) → hash table (muchos campos). ~30% menos memoria que JSON serializado.

T3.2.6 Lists — Listas enlazadas

Las listas son **listas enlazadas** bidireccionales que permiten inserción y eliminación eficiente por ambos extremos. Son ideales para colas (FIFO), pilas (LIFO) y historiales.

Contenido avanzado: La estructura interna quicklist de Redis combina la eficiencia de memoria de listpack con el acceso rápido de lista enlazada. Es transparente para el usuario.

Internamente: listpack vs quicklist

- **listpack:** Cuando la lista es pequeña, Redis usa un listpack (lista compacta en un solo bloque de memoria).
- **quicklist:** Cuando la lista crece, Redis usa una lista doblemente enlazada de listpacks. Es la estructura por defecto desde Redis 3.2+.

Operaciones con Lists

```
# === Insertar por Los extremos ===
LPUSH colas:tareas "tarea1"      # Izquierda (inicio) - O(1)
RPUSH colas:tareas "tarea2"      # Derecha (fin) - O(1)
LPUSHX colas:tareas "tarea0"     # Solo si la lista existe

# === Extraer por Los extremos ===
LPOP colas:tareas                # Izquierda → FIFO
RPOP colas:tareas                # Derecha → LIFO
LPOP colas:tareas 3              # Extraer 3 elementos de la izquierda

# === Cola FIFO bloqueante (espera hasta que haya datos) ===
LPUSH colas:email "email1"
LPUSH colas:email "email2"
BRPOP colas:email 0              # Bloquea indefinidamente
BRPOP colas:email 30             # Bloquea máximo 30 segundos

# === Rangos y acceso por índice ===
LRANGE colas:tareas 0 -1         # Todos los elementos
LRANGE colas:tareas 0 9          # Primeros 10 elementos
LINDEX colas:tareas 0            # Elemento en posición 0
LINDEX colas:tareas -1           # Último elemento

# === Modificar elementos ===
LSET colas:tareas 0 "nueva_tarea" # Cambiar elemento en posición 0
LINSERT colas:tareas BEFORE "tarea1" "tarea0" # Insertar antes de "tarea1"

# === Longitud y eliminación ===
LLEN colas:tareas                # Longitud de la lista
LTRIM colas:tareas 0 99           # Mantener solo los primeros 100 elementos
LREM colas:tareas 2 "tarea1"      # Eliminar 2 ocurrencias de "tarea1"
```

BRPOP/BLPOP = Blocking Pop. El cliente entra en estado bloqueado en la red: no consume CPU ni memoria. Redis lo despierta cuando hay datos en la lista, o tras el timeout. Es el patrón productor-consumidor: el productor hace LPUSH, el consumidor hace BRPOP y espera pacientemente. Más eficiente que un polling cada 100ms.

Analogía: Es como la caja de un supermercado: el cajero pone la siguiente compra en la cinta (LPUSH). El cliente espera hasta que haya una compra en la cinta (BRPOP). Si la cinta está vacía, el cajero no procesa nada; en cuanto llega algo, empieza a trabajar.

Twitter usa listas para colas de procesamiento de tweets y notificaciones push. Cada tweet pendiente de procesar se añade con LPUSH a una cola, y los workers lo procesan con BRPOP. Esto garantiza que cada notificación se procese exactamente una vez y permite múltiples workers en paralelo.

Resumen Lists: Listas enlazadas bidireccionales. LPUSH/RPUSH para insertar, LPOP/RPOP para extraer. BRPOP para colas bloqueantes (FIFO). Internamente: listpack → quicklist.

T3.2.7 Sets — Conjuntos únicos

Los sets son **conjuntos desordenados** de strings únicos. No permiten duplicados y soportan operaciones de teoría de conjuntos (intersección, unión, diferencia).

Contenido avanzado: Redis elige automáticamente la estructura interna según el tamaño y tipo de datos. No necesitas configurar nada, pero es útil saber que conjuntos pequeños de enteros son especialmente eficientes en memoria (intset).

Internamente: listpack vs intset vs hash table

- **intset:** Cuando todos los elementos son enteros y el conjunto es pequeño. Es la estructura más eficiente en memoria.
- **listpack:** Cuando el conjunto es pequeño y los elementos son strings cortos.
- **hash table:** Cuando el conjunto crece. Usa una hash table clásica.

¿Qué es un Intset? Array contiguo de enteros ordenados. Búsqueda binaria $O(\log N)$. Cuando crece demasiado, se convierte en hash table. Para conjuntos pequeños de enteros (hasta ~ 128), Intset es más eficiente que una hash table completa porque no usa punteros por elemento.

```
# === Añadir miembros ===
SADD usuarios:online "user:1000"
SADD usuarios:online "user:1001"
SADD usuarios:online "user:1002"
SADD usuarios:online "user:1000"    # Ignorado (duplicado)

# === Verificar membresía ===
SISMEMBER usuarios:online "user:1000"    # 1 (true)
SISMEMBER usuarios:online "user:9999"    # 0 (false)
SMISMEMBER usuarios:online "user:1000" "user:9999"    # [1, 0]

# === Obtener miembros ===
SMEMBERS usuarios:online                # Todos los miembros
SRANDMEMBER usuarios:online 3           # 3 miembros aleatorios
SCARD usuarios:online                  # Número de miembros (3)

# === Eliminar miembros ===
SREM usuarios:online "user:1000"

# === Operaciones de conjuntos ===
SADD likes:post:1 "user:1000" "user:1001" "user:1002"
SADD likes:post:2 "user:1001" "user:1002" "user:1003"

# Intersección (likes en AMBOS posts)
SINTER likes:post:1 likes:post:2    # user:1001, user:1002

# Unión (likes en CUALQUIER post)
SUNION likes:post:1 likes:post:2    # user:1000, user:1001, user:1002, user:1003

# Diferencia (likes solo en post:1)
SDIFF likes:post:1 likes:post:2    # user:1000

# Almacenar resultado en una nueva clave
SINTERSTORE communes likes:post:1 likes:post:2
```

GitHub usa sets para rastrear usuarios que han dado "star" a un repositorio. `SADD repo:456:stars "user:1000"` añade un star. `SCARD repo:456:stars` cuenta el total. `SINTER repo:456:stars repo:789:stars` encuentra usuarios que han dado star a ambos repositorios.

Resumen Sets: Conjuntos sin duplicados. Soportan `SINTER` (intersección), `SUNION` (unión), `SDIFF` (diferencia). Internamente: `intset` (solo enteros) → `listpack` → `hash table`.

T3.2.8 Sorted Sets — Conjuntos ordenados

Los sorted sets son como los sets pero con una **puntuación (score)** asociada a cada miembro. Los miembros se mantienen ordenados por puntuación de menor a mayor. Son ideales para rankings, leaderboards y timelines.

Contenido avanzado: La skip list permite búsquedas $O(\log N)$ manteniendo orden, y la hash table permite acceso directo $O(1)$ por miembro. Redis elige automáticamente según el tamaño.

Internamente: listpack vs skip list + hash table

- **listpack:** Cuando el sorted set es pequeño (< 128 miembros y $\text{scores} < 64$ bytes).
- **skip list + hash table:** Cuando el sorted set crece. La skip list permite rangos $O(\log N)$ y la hash table permite acceso directo $O(1)$ por miembro.

¿Cómo funciona internamente una Skip List? Imagina una lista enlazada ordenada. Ahora añade "atajos": algunos elementos tienen punteros a elementos 2 posiciones adelante, otros a 4 posiciones, etc. Para buscar, empiezas por el nivel más alto y saltas grandes distancias; cuando te pasas, bajas un nivel y haces saltos más pequeños. Resultado: búsqueda $O(\log N)$ con implementación mucho más simple que un árbol balanceado.

Nivel 0 (lista completa)



Nivel 1 (atajos medios)



Nivel 2 (atajos largos)



```
# === Añadir miembros con puntuación ===
ZADD leaderboard 1500 "player:1000"
ZADD leaderboard 2300 "player:1001"
ZADD leaderboard 1800 "player:1002"
ZADD leaderboard 2300 "player:1003"

# Añadir múltiples miembros de una vez
ZADD leaderboard 2100 "player:1004" 1900 "player:1005"

# === Obtener rango (de menor a mayor) ===
ZRANGE leaderboard 0 -1 WITHSCORES      # Todos con puntuación
ZRANGE leaderboard 0 2 WITHSCORES        # Top 3 (menor a mayor)
ZREVRANGE leaderboard 0 2 WITHSCORES      # Top 3 (mayor a menor)

# === Puntuación y rango de un miembro ===
ZSCORE leaderboard "player:1000"         # 1500
ZRANK leaderboard "player:1000"          # Posición (0-based, de menor a mayor)
ZREVRANK leaderboard "player:1000"       # Posición de mayor a menor

# === Incrementar puntuación ===
ZINCRBY leaderboard 100 "player:1000"    # 1600
ZINCRBY leaderboard -50 "player:1001"    # 2250

# === Rangos por puntuación ===
ZRANGEBYSCORE leaderboard 1000 2000      # Entre 1000 y 2000
ZRANGEBYSCORE leaderboard 1000 2000 LIMIT 0 5 # Primeros 5
ZREVRANGEBYSCORE leaderboard 2000 1000    # De mayor a menor

# === Eliminar ===
ZREM leaderboard "player:1005"
ZREMRANGEBYSCORE leaderboard 0 1000      # Eliminar scores < 1000
ZREMRANGEBYRANK leaderboard 0 2          # Eliminar Los 3 primeros

# === Contar ===
ZCARD leaderboard                        # Número total de miembros
ZCOUNT leaderboard 1000 2000           # Miembros entre 1000 y 2000
```

Twitter almacena timelines como Sorted Sets (timeline:{user_id} → {timestamp: post_id}), manteniendo los 800 posts más recientes por usuario. Cada tweet de un usuario con 10 millones de seguidores se replica a 10 millones de Sorted Sets en menos de un segundo. El score es el timestamp, lo que permite obtener los posts más recientes con ZREVRANGE.

Conexión con T7: En T7 veremos que Netflix modela el historial de reproducción con tablas particionadas en Cassandra, no con Sorted Sets. Cada tecnología tiene su mejor caso de uso según la escala y el patrón de acceso.

Resumen Sorted Sets: Conjuntos ordenados por score. Ideales para rankings (ZADD, ZREVRANGE). Internamente: listpack → skip list + hash table. $O(\log N)$ por operación.

Caso real: Twitter — Métricas y Timeline Cache

Twitter usa Sorted Sets para timelines — ver T3.1 para métricas detalladas.

Caso real: Instagram — Feed Híbrido y Métricas

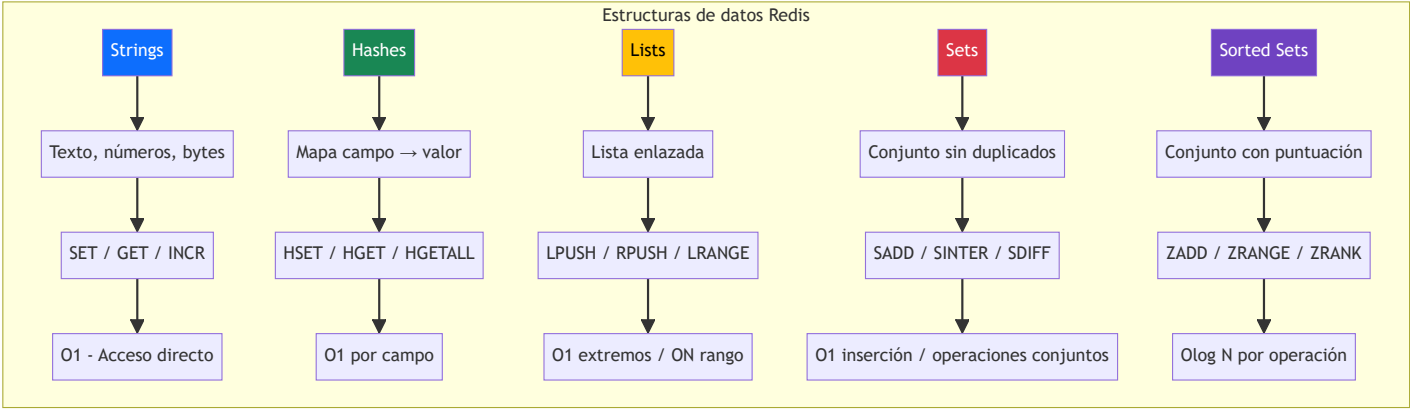
Instagram usa un **patrón híbrido** para generar el feed de noticias:

- Usuarios normales (< 100K seguidores):** fan-out on write. Cada post se copia a los timelines de todos sus seguidores usando Redis Sorted Sets.
- Celebridades (> 100K seguidores):** fan-out on read. El feed se genera al vuelo consultando Cassandra, sin replicar a Redis.

Métrica	Valor
Usuarios activos diarios	500 millones
Lecturas de feed/día	2.500 millones
Almacenamiento Redis	~1,2 TB (40 nodos × 32 GB)
Uploads de fotos/vídeos/día	100 millones

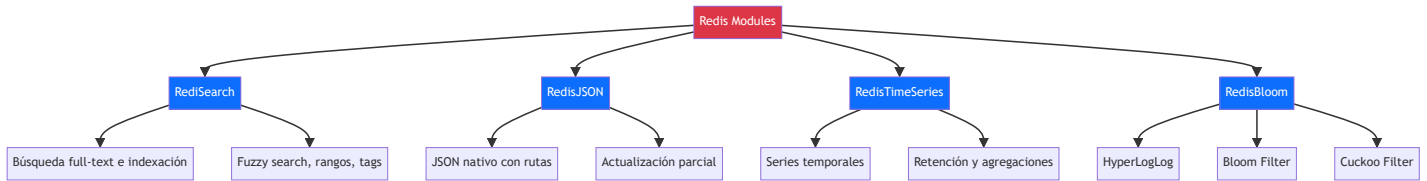
Lección: No existe una solución única para feeds. Instagram combina fan-out on write (para usuarios con pocos seguidores) y fan-out on read (para celebridades) para equilibrar rendimiento y consumo de memoria.

T3.2.9 Resumen visual de estructuras de datos



T3.2.10 Redis Modules — Ampliando las capacidades

Redis no es solo una caché de clave-valor. Desde Redis 5+ con módulos, es una plataforma completa:



RediSearch — Búsqueda full-text e indexación

Crea índices sobre hashes y JSON para búsquedas full-text, fuzzy, por rango numérico y por tags. Sustituye la necesidad de Elasticsearch para casos simples.

Ejemplo: RediSearch

```
# Crear índice
FT.CREATE idx:products ON HASH PREFIX 1 product: SCHEMA
  name TEXT
  price NUMERIC
  category TAG

# Indexar productos
HSET product:1 name "iPhone 15" price 999 category "phones"
HSET product:2 name "Samsung Galaxy" price 899 category "phones"
HSET product:3 name "MacBook Pro" price 2499 category "laptops"

# Búsqueda full-text
FT.SEARCH idx:products "iPhone"

# Búsqueda por rango de precio
FT.SEARCH idx:products "@price:[800 1000]"

# Búsqueda por tag
FT.SEARCH idx:products "@category:{phones}"
```

RedisJSON — JSON nativo

Almacena y consulta JSON sin serializar/deserializar. Soporta rutas JSON y actualización parcial.

```
# Almacenar JSON
JSON.SET user:1000 $ '{"name":"Juan","address":{"city":"Madrid","zip":"28001"}}'

# Consultar con rutas JSON
JSON.GET user:1000 $.name           # "Juan"
JSON.GET user:1000 $.address.city    # "Madrid"

# Actualización parcial
JSON.SET user:1000 $.address.city "Barcelona"
```

RedisTimeSeries — Series temporales

Datos de series temporales con retención automática y agregaciones (avg, sum, min, max). Ideal para métricas, IoT y monitoreo.

RedisBloom — Filtros probabilísticos

- **HyperLogLog:** Contar elementos únicos con 0,81% de error y solo 12 KB de memoria. Ideal para contar visitantes únicos, usuarios activos, etc.
- **Bloom Filter:** Verificar pertenencia a un conjunto sin almacenar cada elemento. Útil para evitar consultas a BD para datos que no existen.
- **Cuckoo Filter:** Alternativa a Bloom con soporte de eliminación.

Instagram usa HyperLogLog para contar usuarios únicos que ven una Story sin almacenar cada usuario. Con solo 12 KB de memoria por Story, consiguen precisión del 99,2% para counts de hasta 4 mil millones de elementos únicos.

Resumen Módulos: RediSearch (búsqueda full-text), RedisJSON (JSON nativo), RedisTimeSeries (series temporales), RedisBloom (HyperLogLog, Bloom Filter). Amplían Redis más allá de clave-valor.

Cuando lo pongas en práctica

- Elige la estructura de datos ANTES de escribir datos — cambiar después es costoso y requiere migración.
- Usa hashes para objetos con múltiples campos (ahorra ~30% de memoria frente a strings serializados).
- Para IDs únicos, INCR es más simple y eficiente que generar UUIDs.
- Sorted Sets son ideales para rankings y feeds temporales — el score como timestamp resuelve el orden cronológico.
- Prueba `MEMORY USAGE` key para medir cuánta memoria consume cada clave y optimizar el modelado.

Preguntas para reflexionar

Pregunta 1

Si un usuario tiene 5 campos y siempre se leen todos juntos, ¿qué estructura usarías? ¿Y si cada campo necesita un TTL diferente?

Pregunta 2

¿Por qué Twitter usa Sorted Sets para timelines en lugar de Listas? ¿Qué ventaja tiene poder ordenar por timestamp y acceder por rango?

Pregunta 3

Si un post de Instagram tiene 10 millones de likes, ¿qué estructura de datos usarías para almacenarlos? Justifica tu respuesta considerando memoria, rendimiento de escritura y lectura.



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid